TECHNISCHE UNIVERSITÄT
ILMENAU

Institut für Technische Informatik und Ingenieurinformatik
Fakultät für Informatik und Automatisierung
Fachgebiet System- und Software-Engineering

Bachelorarbeit

Modellgetriebene Microservice-Entwicklung

vorgelegt von

Patrik Löffler
Matrikelnummer 56207

Betreuer:

Prof. Armin Zimmermann
Dr. Ralph Maschotta, Tino Jungebloud

Ilmenau, den 31. Oktober 2023

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Einführung in das Thema 1

1.2 Zielstellung 1

2 Grundlagen **3**

2.1 Domain-Driven Design 3

2.1.1 Definition 3

2.1.2 Bausteine des Model-Driven Design 4

2.1.3 Strategic Design 7

2.2 Microservices 11

2.2.1 Definition 11

2.2.2 Architektur von Microservice-Systemen 12

2.2.3 Kommunikation 14

2.2.4 Betrieb und Continious Delivery 15

2.3 Cloud-Infrastruktur 16

2.3.1 Definition 16

2.3.2 Docker 16

2.3.3 Kubernetes 16

2.4 Modellgetriebene Entwicklung von Microservices 18

2.5 Noch zu sichten 19

2.5.1 ARGON - automated cloud infrastructure provisioning using a domain-specific language 19

2.5.2 model-driven approach for adaptive microservice-based systems using VxBPMN4MS 19

2.5.3 Context Mapper, a tool for DDD and microservice design. 20

2.5.4 DOML simplifies infrastructure deployments by unifying multiple IaC languages 20

1 Einleitung

1.1 Einführung in das Thema

1.2 Zielstellung

Ziel der Arbeit ist es, Anwendungen, die das Microservice-Paradigma verwenden, mit Werkzeugen und Methoden der modellgetriebenen Softwareentwicklung zu entwickeln.

Dazu wird ein Metamodell konzipiert, welches die grundlegenden Eigenschaften des Paradigmas abstrahiert. Ergänzt wird dies durch die Berücksichtigung von Konzepten des Domain-Driven Design zur Modellierung der Geschäftslogik.

Die mithilfe dieses Metamodells generierten Anwendungen sollen in der Lage sein, bereits existierende, sogenannte Microservice Chassis – wiederverwendbare Grundbausteine eines Microservices – zu verwenden. Daraufhin wird erforscht, welche Möglichkeiten es gibt, diese generierten Anwendungen automatisiert, in Abhängigkeit von Versionsaktualisierungen der Chassis, zu migrieren.

Außerdem wird untersucht, inwiefern die der Anwendung zugrunde liegenden IT-Infrastruktur in solch einem Metamodell berücksichtigt werden kann und welche Möglichkeiten sich daraus ergeben.

2 Grundlagen

2.1 Domain-Driven Design

2.1.1 Definition

„Domain-Driven Design“ (Abk. DDD) ist eine Herangehensweise, Problemdomänen in ein Modell für die Softwareentwicklung zu überführen.

Maßgeblich geprägt wurde das DDD durch Eric Evans in seinem Buch „Domain-Driven Design: Tackling Complexity in the Heart of Software“ [Eva07]. Kern des domänengetriebenen Entwurfs ist es, die reale, oft unübersichtliche Fachlichkeit fortschreitend in feiner geschnittene Teilbereiche zu kapseln. Diese sollten weiterhin möglichst wenige Abhängigkeiten zueinander haben [Dow18, S.63, 70]. Dieser auf das Domänenmodell fokussierte Ansatz versucht dabei, Kernaspekte einer Anwendung möglichst im Modell zu integrieren, anstatt sie über externe Dienste zu binden [Das17, S.74].

Evans stellt drei grundlegende Eigenschaften fest, welche bei der Wahl eines Modells im DDD zu berücksichtigen sind [Eva07, S. 3-4]:

- Es existiert immer eine Abhängigkeit zwischen Modell und Implementierung.
- Die Terminologie während der Softwareentwicklung ist eine Implikation des zugrunde liegenden Modells.
- Nützliches und aufgearbeitetes Wissen kann diesem Modell entnommen werden.

Ein weiteres Kernkonzept des DDD ist die „Ubiquitous Language“. Sie verbindet Entwickler und Domänen-Experten durch eine auf dem Modell basierenden Sprache. Dabei definiert sie eindeutig die verwendeten Begriffe und die ihnen zugeordneten Fachlichkeiten [Eva07, S. 25-27]. Ein weiterer Kern ist das „Model-Driven Design“ (Abk. MDD). MDD betont und bezeichnet das notwendige Ineinandergreifen der Konzeption eines Domänenmodells mit dem Entwurf der Anwendungssoftware. Um eine enge Übereinstimmung zwischen Modell und Software sicherzustellen, ist es von Vorteil, innerhalb eines durch Software-Tools unterstützten Modellierungsansatzes zu arbeiten. Objektorientierte Programmiersprachen eignen sich besonders gut, da sich die objekthaften Eigenschaften von Modellen gut abbilden lassen [Eva07, S. 49-51].

2.1.2 Bausteine des Model-Driven Design

Das DDD definiert verschiedene Bausteine des MDD [Eva07, S. 65]:

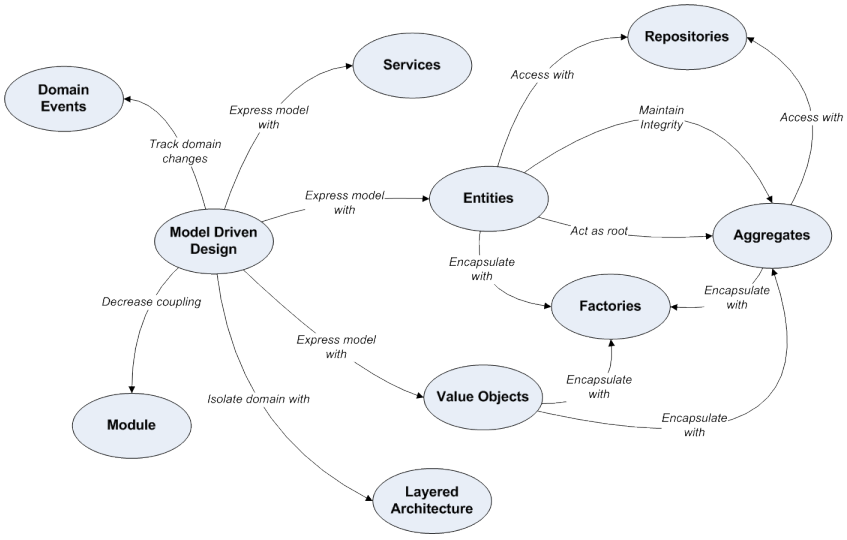

Abbildung 2.1: Bausteine des Model-Driven Design

Schichtenarchitektur

Schichtearchitekturn helfen die Komplexität von Software zu beherschen. Jedes Element einer Schicht hängt nur von Elementen derselben oder einer tieferen Schicht ab. Beim DDD wird die Geschäftslogik in eine eigene Schicht, die „Domain Layer“, gelegt. Dies sorgt für eine enge Verbindung zum Modell, wie es das MDD vorsieht. Mit Mustern wie dem Model-View-Controller kann man dann diese Schicht nutzen [Eva07, S. 69 - 73].

Assoziationen im Modell

Wie Assoziationen zwischen Modell und Umsetzung dargestellt werden, kann sich unterscheiden. Man kann diese vereinfachen, indem man z.B. eine bestimmte Richtung festlegt, was das Design klärt und Abhängigkeiten verringert. Ein weiterer Weg ist das Hinzufügen eines Qualifikators, um die Anzahl der Beziehungen zu reduzieren. Zudem kann man unwichtige Assoziationen auch ganz weglassen [Eva07, S. 81-85].

Entities

„Entities“ sind identifizierbare Objekte eines speziellen Bereichs. Wichtig ist, dass man sie einzeln identifizieren kann, nicht ihre spezifischen Merkmale. Dies kann z.B. mit automatisch erzeugten Identifikationswerten geschehen. Sie symbolisieren die durchgehende Existenz eines Objekts über dessen Lebenszyklus. Beim Erstellen eines Modells geht es hauptsächlich darum, diese

Kontinuität zu gewährleisten. Klassendetails, Aufgaben und Assoziationen sollten die Identität der Entity zeigen. Entitäten sollen sparsam verwendet werden. Nur notwendiges Verhalten soll ihnen hinzu gefügt werden. Zusätzliches Verhalten soll zu anderen zugehörigen Objekten verschoben werden [Eva07, S. 89-96] [Das17, S.75].

Value Objects

„Value Objects“ sind Typen ohne eigene Identität, die spezifische Werte repräsentieren. Sie können aus Value Objects Objekten bestehen und Entitäten referenzieren. Oft werden sie als Parameter zwischen Objekten oder als Eigenschaften von Entitäten genutzt. Value Objects sollten ein stimmiges Ganzes bilden und sind idealerweise unveränderlich, wodurch sie sicher geteilt werden können. Bidirektionale Verbindungen bei Value Objects sind aufgrund ihrer fehlenden Identität nicht sinnvoll und sollten vermieden werden [Eva07, S. 97-103] [Das17, S.75].

Services

„Services“ bieten eine spezielle Funktion im Modell an, ohne dabei Daten oder Zustände zu speichern. Er wird nach dem benannt, was er macht, und nicht nach dem, was er ist. Services starten oft bestimmte Vorgänge und arbeiten mit den Objekten des Modells. Sie kombinieren Schritte in Geschäftsprozessen. Sie sollten bedacht eingesetzt werden und dabei nicht alle Aufgaben von Entities und Value Objects abnehmen.

Ein guter Service sollte folgende Eigenschaften erfüllen:

- Er bezieht sich auf ein Konzept, das nicht direkt zu einer Entität oder einem Wertobjekt gehört.
- Es existiert eine klare Schnittstelle im Bezug auf andere Elemente des Modells.
- Kein Zustand wird gespeichert

Da Services auch außerhalb der Domain Layer auftreten können ist eine präzise fachliche Betrachtung nötig [Eva07, S. 104 - 108] [Das17, S.74].

Packages

„Packages“ sorgen im Modell für starke Kohäsion innerhalb und schwache Kopplung zwischen ihnen, indem sie zusammengehörige Elemente bündeln [Eva07, S. 109 - 116].

Aggregates

„Aggregates“ sind Ansammlungen von Objekten, die zusammen als Einheit betrachtet werden. Ihr Zweck ist es, Veränderungen als Ganzes zu handhaben, um Konsistenz zu gewährleisten. Ein direkter Zugriff auf einzelne Bestandteile kann Probleme verursachen. Deshalb gibt es in einem Aggregat ein Hauptobjekt (Wurzel), das für alle Aktionen verwendet wird.

In jedem Aggregat gibt es diese Hauptobjekt und einen definierten Bereich, in dem sich alle zugehörigen Objekte befinden. Andere Objekte können nur auf dieses Hauptobjekt verweisen, wobei die enthaltenen Objekte untereinander verweisen können. Objekte innerhalb des Aggregats, die nicht die Hauptfunktion ausüben, sind nur in ihrem Kontext eindeutig identifizierbar [Eva07, S. 123 - 135] [Das17, S.76].

Factories

Eine Factory erstellt Objekte. Dies ist nützlich, wenn das Erstellen eines Objekts mehr ist als nur das Ausführen eines Konstruktors, da manchmal besondere Regeln oder Logik benötigt werden. Zwei grundlegende Anforderungen an einer Factory sind:

- Sie stellt sicher, dass jedes erstellte Objekt korrekt und vollständig ist.
- Sie fokussiert sich auf den Typ des Objekts, das erstellt werden soll, und nicht unbedingt darauf, wie es gemacht wird.

Auch wenn eine Factory dafür sorgt, dass ein Objekt richtig erstellt wird, kann sie einige Überprüfungen an andere Teile auslagern. Es gibt auch Factories, die bestehende Objekte wiederherstellen. Dabei gibt es Unterschiede, z.B. weisen sie keine neuen IDs zu und sie müssen Verletzungen von Regeln anders behandeln [Eva07, S. 136 - 146] [Das17, S.77].

Repositories

„Repositories“ sind dafür zuständig, Daten von bestimmten Objekten, wie Entitäten oder Aggregaten, sicher zu speichern und zu verwalten. Die Hauptidee von Repositories ist, einen zentralen Ort zu haben, der dafür sorgt, dass alles korrekt und konsequent gespeichert wird. Man kann sich ein Repository wie eine besondere Sammlung vorstellen, aus der man Daten anhand von bestimmten Kriterien abfragen kann. Dabei fügt das Repository neue Objekte der Datenbank hinzu oder löscht sie. Wenn man Daten aus einem Repository möchte, nutzt man spezielle Suchmethoden. Dabei kann die Suche einfach oder auch sehr detailliert sein [Eva07, S. 147 - 161] [Das17, S.76].

2.1.3 Strategic Design

Als „Strategic Design”wird im DDD beschrieben, wie verschiedene Domänenmodelle miteinander interagieren können. Dabei müssen die strategischen Designprinzipien die Designentscheidungen leiten, um die Abhängigkeiten der Teile zu verringern und die Klarheit zu erhöhen, ohne dabei die kritische Interoperabilität und Synergie zu verlieren [Wol16, S. 45] [Eva07, S. 334].

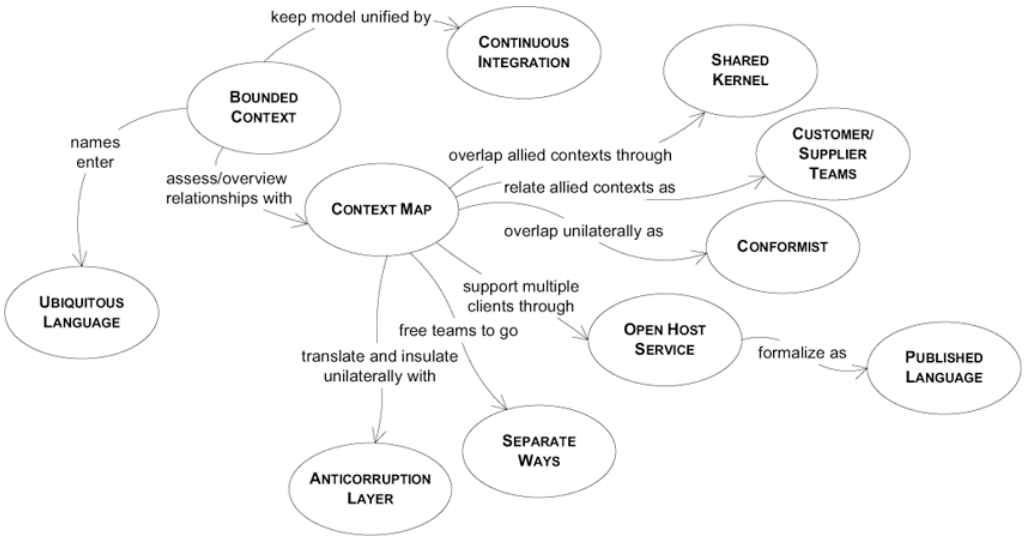

Abbildung 2.2: Supple Design

Bounded Context

Im Zentrum dieser Designentscheidungen steht der „Bounded Context“. Der Grundgedanke dahinter ist, dass jedes Domänenmodell nur in einem klar definierten Rahmen sinnvoll ist [Wol16, S. 45]. Dieser Rahmen könnte ein spezifischer Codebereich oder die Arbeit eines Teams sein. Der ”Bounded Context”beschreibt die Bedingungen, unter denen die Begriffe im Modell eine bestimmte Bedeutung haben. Eine Domäne kann aus mehreren solchen Kontexten bestehen, die auch Informationen mit anderen Kontexten teilen können. Jeder Bounded Context hat eine Schnittstelle, die bestimmt, welche Modelle für andere Kontexte verfügbar sind [New15, S.57].

Ein Bounded Context legt fest, in welchem Bereich ein Modell und somit auch die dazugehörige Ubiquitous Language gültig ist. Das hilft dabei, Verknüpfungen und mögliche Missverständnisse an Schnittstellen zu erkennen. Ein solcher Kontext klärt das Verständnis eines Modells innerhalb eines Teams und wie es sich zu anderen Kontexten verhält. Im Rahmen dieses Kontexts sollte das Modell konsistent gehalten werden, ohne dass man sich Sorgen um die Relevanz außerhalb dieser Grenzen machen muss [Eva07, S. 335 - 340] [Dow18, S.64-65].

Context Map

Eine „Context Map“ liegt an der Schnittstelle zwischen Projektmanagement und Softwareentwurf. Sie ist eine Übersicht über bestehende Modelle, wobei der Fokus auf den Berührungspunkten liegt. Diese Karte macht explizit die Kommunikationswege und gemeinsam genutzten Bereiche deutlich. Context Maps werden verwendet, um die Beziehungen verschiedener Bounded Contexts darzustellen [Eva07, S. 344 - 353] [Wol16, S. 45].

Shared Kernel

Ein Bereich des Domänenmodells kann dazu dienen, zwischen Teams geteilt zu werden. Das betrifft sowohl die Codebasis als auch die Datenbank, die mit dem Modell verknüpft sein könnte. Bei einem solchen „Shared Kernel“ teilen sich die Domänenmodelle einige Elemente, können sich jedoch in anderen Aspekten unterscheiden. Dies kann entweder implizit geschehen oder explizit durch den gemeinsamen Codegebrauch bzw. durch den Zugriff auf dieselben Tabellen in der Datenbank. Oft bezeichnet der Shared Kernel die Kern-Domäne, eine Sammlung generischer Teilbereiche oder beides. Ziel dabei ist es, Wiederholungen zu vermeiden [Eva07, S. 354 - 355] [Wol16, S. 45] [Dow18, S.65].

Customer/Supplier

Zwischen Parteien mit zwei verschiedenen Kontexten kann eine Beziehung entstehen, die aus einer nachgelagerten und einer vorgelagerten Komponente besteht. Das Domain-Driven Design definiert deshalb eine klare Kunden-/Lieferantenbeziehung für solche Kontexte. Die „Customer/Supplier“-Beziehung zeigt, dass ein Untersystem dem Aufrufer ein Domänenmodell zur Verfügung stellt. Der Aufrufer, also der Kunde, legt die Struktur des Modells fest. Der Anbieter der Schnittstelle richtet sich beim Design nach den Wünschen des Kunden und gibt ihm oft ein Mitspracherecht. Es sind automatisierte Tests und Überwachungen notwendig, um Schäden im nachgelagerten Bereich zu vermeiden [Eva07, S. 356 - 360] [Wol16, S. 45,47] [Dow18, S.66].

Conformist

Wenn die Anforderungen eines nachgelagerten, abhängigen Subsystems durch den vorgelagerten Teil gedeckt werden können, bietet es sich an, dem nachgelagerten Teil die Rolle des „Conformist“ zuzuweisen. Dies vereinfacht den Datenaustausch zwischen den beiden Kontexten, indem beide dasselbe Modell verwenden. Der Aufrufer nutzt also das gleiche Modell wie das aufgerufene System und passt sich somit dem anderen Modell an [Eva07, S. 361 - 363] [Wol16, S. 47] [Dow18, S.66].

Anticorruption Layer

Wenn man Systeme kombiniert, die auf verschiedenen Modellen basieren, kann es passieren, dass das Modell des einen Systems durch das andere beeinflusst wird. Ein „Anticorruption Layer“ kann helfen, indem er eine Schutzschicht bildet. Diese Schicht ermöglicht es den Nutzern, mit ihrem gewohnten Modell zu arbeiten, ohne das andere System zu beeinflussen. Sie dient als Übersetzer zwischen den beiden Modellen und sorgt dafür, dass sie unabhängig voneinander bleiben. Die Schicht kommuniziert über eine Schnittstelle mit dem anderen System, ohne dass große Änderungen daran notwendig sind. Sie kann bei Bedarf in beide Richtungen zwischen den Modellen übersetzen [Eva07, S. 364 - 370] [Wol16, S. 47] [Dow18, S.66].

Separate Ways

Wenn der Nutzen im Vergleich zu den Integrationskosten zu gering ist, kann es sinnvoll sein, festzulegen, dass ein bestimmter Kontext keine Verbindung zu anderen hat. Im DDD spricht man hier von „Separate Ways“, was bedeutet, dass die beiden Systeme nicht verbunden werden und unabhängig voneinander bleiben. Anstatt Lösungen für Probleme gemeinsam zu nutzen, müssen diese auf einem klaren, speziellen Weg innerhalb dieses Rahmens umgesetzt werden. Dies kann zum Beispiel mit dem Composite UI Pattern erreicht werden [Eva07, S. 371 - 373] [Wol16, S. 47] [Dow18, S.66].

Open Host Service

Wenn ein Teilsystem mit vielen anderen verbunden werden muss, kann es problematisch sein, für jedes einen Übersetzer anzupassen. Hier bietet es sich an, ein offenes Protokoll von Diensten zu definieren, das Zugang zum Teilsystem ermöglicht. Dieser Kontext, als „Open Host Service“ bezeichnet, stellt also Schnittstellen bereit, die jeder nutzen kann. So kann jeder Bereich seine eigene Anbindung erstellen. [Eva07, S. 374] [Wol16, S. 48] [Dow18, S.66].

Published Language

Das direkte Übersetzen von Modellsprachen kann zu sehr komplexen Strukturen führen. Selbst wenn eine Sprache für den Datenaustausch genutzt wird, ist sie einmal festgelegt und lässt sich nicht leicht an Entwicklungsbedürfnisse anpassen. Deshalb kann eine gut dokumentierte, gemeinsame Sprache als allgemeines Kommunikationsmittel bereitgestellt werden. Diese „Published Language“stellt eine Modellierung der Domäne als universelle Sprache zwischen den Bounded Contexts dar [Eva07, S. 375-378] [Wol16, S. 48] [Dow18, s.65].

2.2 Microservices

2.2.1 Definition

Der zur Modularisierung eingesetzte Ansatz der Microservices erfüllt verschiedene Grundlegende Vorraussetzungen: In ihrer Implementierung gibt es keine technologischen Einschränkungen. Die Dienste können unabhängig voneinander überarbeitet und auf ein Zielsystem ausgerollt werden. Weiterhin sollen Build und Deployement automatisiert sein. Sie besitzen einen eigenen Datenhaushalt sowie eigenständige Prozesse auf der ausführenden Infrastruktureinheit. Microservices nutzen verteilte Netzwerkkommunikation, um sich mit anderen Diensten zu verständigen. Transaktionen zeichnen sich durch die ACID-Eigenschaften aus, zu denen Atomizität, Konsistenz, Isolation und Dauerhaftigkeit gehören. Es ist wichtig, dass Daten konsistent bleiben. Deshalb sollten keine Transaktionen über mehrere Services hinweg durchgeführt werden. Um diese Konsistenz zu gewährleisten, können Kompensationstransaktionen eingesetzt werden. [Wol16, S. 2] [Wol16, S. 31-37]

Jeder Microservice soll eine eigene fachliche Einheit bilden, die so geschnitten ist, dass für Änderungen oder neue Features nur ein Microservice geändert werden muss. Weiterhin sollen die aus dem DDD bekannten Konzepte des Strategic Design verwendet werden. Micorservices sollen jeweils eine fachliche Domäne modellieren, sodass neue Features oder Änderungen nur in einem Microservice umgesetzt werden müssen. Ein Bounded Context kann in mehrere Microservices aufgeteilt werden, wenn das sinnvoll erscheint. [Wol16, S. 48-51]

Damit Microservices unabhängig voneinander in Produktion gebracht werden können, ist eine gekapselte Hardware nötig, welche durch Virtualisierung umgesetzt werden kann. Dies setzt eine passende Infrastruktur voraus, die automatisiert eine große Anzahl virtueller Maschinen erzeugen kann. Weiterhin benötigt jeder Microservice, als Folgerung aus der Voraussetzung, unabhängig deployen zu können, eine eigene Continuous-Delivery-Pipeline. Zudem muss es ein umfassendes Monitoring geben, da durch die Vielzahl an Anwendungen sonst Probleme nicht erkannt werden können. Dieses sollte neben technischen Metriken auch Anwendungsmetriken erfassen, um eine umfassendere Problemübersicht zu bekommen. Auch eine unabhängige Versionskontrolle der Dienste ist notwendig, da aus der unabhängigen Deploybarkeit auch verschiedene Versionierungen möglich sind. [Wol16, S. 80-81]

2.2.2 Architektur von Microservice-Systemen

Gemäß den DDD-Prinzipien kann eine fachliche Architektur erfolgen. Dabei soll es zwischen den Microservices eine lose Kopplung geben, wodurch sie isoliert und leichter abänderbar sind. Innerhalb eines Dienstes sollen die Bestandteile eng zusammenarbeiten, was einer hohen Kohäsion entspricht. Weiterhin sind zyklische fachliche Abhängigkeiten in der Architektur problematisch, da dadurch die isolierte Änderbarkeit nicht mehr gewährleistet ist. [Wol16, S. 101-105]

Verschiedene etablierte Methoden existieren im Zusammenhang mit Änderungen der Gesamtarchitektur, die durch neue Erkenntnisse aus der Domäne notwendig sein können. So kann in mehreren Diensten gemeinsam verwendeter Code in Bibliotheken ausgelagert werden. Dabei muss jedoch auf die dadurch entstehende Abhängigkeit geachtet werden, da durch unkoordinierte Änderungen Fehler in Teilen des Microservice-Systems einschleichen können. Weiterhin kann Code eines Dienstes in einen anderen übertragen werden, um Verbesserungen im Hinblick auf die Kohäsion in einem Dienst zu erreichen. Eine Alternative zur Bibliothek ist es, Code in einen gemeinsamen, neuen Dienst zu übertragen, was bei vorausgesetzter passender Schnittstellendefinition dem Ziel der entkoppelten Dienste zuträglich ist. Es ist auch möglich, alten Code in einen neuen Service zu migrieren oder die notwendige Änderung der Architektur mit einem vollständig neu entwickelten Microservice umzusetzen. [Wol16, S. 112-120]

Damit Microservices ihre gemeinsame Logik implementieren können, müssen sie sich gegenseitig aufrufen. Dabei kann eine ereignisgetriebene Architektur (Event-Driven Architecture, EDA) verwendet werden, bei der ein Dienst bei einem Ereignis aus der Anwendungslogik, einem Event, dieses versendet. Dieser Dienst, der Event-Emitter, signalisiert damit allen interessierten Microservices (Event-Consumern), dass etwas passiert ist. Dieses Vorgehen kapselt die Anwendungslogik und entkoppelt die Bearbeitung der Events zeitlich, was dem Ziel einer losen Kopplung beiträgt. [Wol16, S. 137 - 138]

Die Konfiguration eines Microservice-Systems wird mit steigender Anzahl von Services entsprechend aufwendig. Mit Werkzeugen wie Zookeeper, etcd oder Spring Cloud Config werden Key-Value-Stores bereitgestellt, die dies vereinfachen. Jedoch gilt es zu beachten, dass dadurch eine Abhängigkeit zu einem Konfigurationsserver entsteht. Alternativ kann eine Konfiguration auf dem Server des Microservices selbst vorliegen. Dies vermeidet die Notwendigkeit eines zentralen Konfigurationsservers, kann aber auch Herausforderungen in Bezug auf die Synchronisation und Verwaltung der Konfigurationen über mehrere Instanzen hinweg mit sich bringen. [Wol16, S. 141 - 142]

Service Discovery stellt sicher, dass Microservices sich gegenseitig finden können. Dazu muss diese dynamisch sein um gehende und kommende Microservices zu finden. Technolgien für diese sind besipielsweiße DNS (Domain Name Sstem), Eureka, Consul. [Wol16, S. 143 - 146]

Bei hoher Last können Microservices unabhängig skalieren. Dadurch müssen mehrere Instanzen registriert werden, die sich die Last gegen einen Dienst teilen. Um HTTP anfragen in solch

einem Fall zu verteilen wird ein Load Balanecr genutzt, der sich nach außen wie eine einzige Instanz verhält, aber die Anfragen auf mehrere verteilt. Da bei einem einzigen Load Balancer für mehrere Microservices, dieser zum Flaschenhals werden kann, sollte pro Microservice ein Load Balancer existieren. Technologien für Load Balancer existieren nativ in Cloud Umgebungen oder können beispielsweiße mit Apache httpd, nginx oder HAProxy umgesetzt werden. [Wol16, S. 147 - 150]

Die Sicherheitsarchitektur muss einheitlich auf der Gesamtebene des Systems definiert werden, um sicherzustellen, dass ein Zugriff eines Nutzers einheitlich behandelt wird. Dazu gehört es, Authentifizierung und Autorisierung zu gewährleisten. Der einzelne Microservice des fachlich abgebildeten Systems soll keine Authentifizierung übernehmen. Stattdessen wird diese Aufgabe von einem zentralen Server wahrgenommen. Bei der Autorisierung ist ein Zusammenspiel gefragt, da über Benutzergruppen bestimmte Features des Microservices zugewiesen sind. Welche dies sind, soll der Microservice entscheiden. Eine Implementierung dieses Konzepts ist das OAuth2-Protokoll. Dabei erhält ein Nutzer nach erfolgreicher Authentifizierung und Autorisierung ein Access Token, welches genutzt wird, um mit einem Service zu interagieren. Dieses kann weiterhin von einem Microservice an andere Microservices weitergereicht werden, um berechtigte Anfragen zu ermöglichen und die einheitliche Sicherheitsrichtlinie im gesamten System zu gewährleisten. [Wol16, S. 153 - 161]

2.2.3 Kommunikation

Mit REST (Representational State Transfer) können Microservices untereinander Code aufrufen. Es gibt eine Vielzahl von Ressourcen, die über URIs identifiziert werden können. Die Ressourcen können über eine feste Menge von Methoden manipuliert werden, welche z. B. durch HTTP vorgegeben sind. Ressourcen können unterschiedlich repräsentiert werden. Beziehungen zwischen Ressourcen können durch Links angegeben werden (HATEOAS). Der Server in einem REST-System sollte zustandslos sein. Eine RESTful-HTTP-Schnittstelle kann durch einen Cache ergänzt werden. RESTful HTTP ist synchron. Daher können Ausfälle auftreten, denen mit Timeouts entgegengewirkt werden kann, um Fehlerfortpflanzung zu vermeiden. [Wol16, S. 182 - 183]

Messaging-Systeme stellen eine weitere Möglichkeit dar, die Kommunikation zwischen Microservices zu ermöglichen. Sie basieren auf dem Versand von Nachrichten, die selbst bei einem Netzwerkausfall übermittelt werden können, da die Messaging-Systeme sie zwischenspeichern. Eine korrekte Verarbeitung kann durch Initiierung einer erneuten Übertragung im Fehlerfall garantiert werden. Diese Art von Messaging-Architekturen ist asynchron und auf hohe Latenzzeiten ausgelegt. Der Aufruf eines anderen Services blockiert die weitere Verarbeitung nicht, sodass ein Microservice ohne Wartezeit auf eine Antwort weiterarbeiten kann. Der Sender kennt den Empfänger nicht, weil die Nachricht auf einer Queue oder einem Topic landet, für die sich potenzielle Empfänger registrieren. Dies entkoppelt Sender und Empfänger.

Messaging bietet eine Lösung für transaktionale Systeme mit Microservices, wobei das Senden und Empfangen von Nachrichten Teil einer Transaktion ist. Tritt bei der Verarbeitung einer Nachricht ein Fehler auf, werden alle gesendeten Nachrichten storniert und jegliche Änderungen rückgängig gemacht. Die Empfänger können ebenfalls transaktional abgesichert sein. Entscheidend dabei ist, dass sowohl das Senden und Empfangen von Nachrichten als auch die Transaktion auf der Datenbank in einer einzigen Transaktion zusammengefasst werden.

Messaging kann durch verschiedene Technologien wie AMQP, Apache Kafka, JMS oder ATOM-Feeds realisiert werden. [Wol16, S. 183 - 187]

Jeder Microservice kann eine oder mehrere Schnittstellen für andere Microservices bereitstellen. Außerdem kann ein Microservice Schnittstellen nach außen haben, die Interaktionen außerhalb des Systemkontexts ermöglichen. Wenn ein Microservice eine Schnittstelle ändert, sollte er auch noch die alte Schnittstelle unterstützen. Weiterhin sollten Schnittstellen getrennt umgesetzt werden. Um Änderungen an einer Schnittstelle deutlich zu machen, kann eine Versionsnummer genutzt werden. Semantic Versioning definiert dabei eine Semantik für Versionsnummern, wobei das Format MAJOR.MINOR.PATCH lautet. Dabei sind Änderungen der Major-Version nicht rückwärtskompatibel, Änderungen der Minor-Version bieten neue Features, und Patch-Änderungen beheben Fehler (Bugfixes). Die Version sollte nicht in einer URL, sondern besser in einem Header bei einem REST-Request angegeben werden. [Wol16, S. 190 - 192]

2.2.4 Betrieb und Continious Delivery

Mit Logging kann eine Anwendung sehr einfach Informationen darüber bereitstellen, welche Ereignisse aufgetreten sind. Anfragen können oftmals nur durch das Zusammenspiel mehrerer Services behandelt werden. Dies erfordert eine Korrelations-ID, die es ermöglicht, einen Request über alle Log-Einträge der einzelnen Dienste hinweg eindeutig zu identifizieren. Logs können von Microservices zentral über das Netzwerk verarbeitet werden. Dies kann beispielsweise mit Technologien wie Logstash, Elasticsearch und Kibana umgesetzt werden. [Wol16, S. 244 - 249]

Monitoring bezeichnet das Überwachen von Metriken einer Software. Bei Microservice-Architekturen sollten die relevanten Informationen dafür in einem einheitlichen Format vorliegen. Grundlegende Metriken für Microservices können beispielsweise ein Wert sein, der die Verfügbarkeit des Dienstes anzeigt (Alive). Weiterhin kann die Verfügbarkeit in Abhängigkeit von den vom Dienst genutzten Diensten und Ressourcen dargestellt werden (Health). Antwortzeiten, die Anzahl bestimmter Fehler oder die Anzahl von Anfragen sind andere mögliche Metriken. Dabei ist es sinnvoll, eine einheitliche Abfrageschnittstelle für alle Services zu nutzen. Da Microservices dynamisch starten und verschwinden können, muss das Monitoring unabhängig vom einzelnen Dienst erfolgen. Dies kann mit Technologien wie Graphite, Grafana oder Nagios umgesetzt werden. Auch in Cloud-Umgebungen stehen entsprechende Werkzeuge zur Verfügung. [Wol16, S. 250 - 256]

Manuelle Deployments sind bei einer Microservice-Architektur nicht sinnvoll umsetzbar. Daher muss dies automatisiert werden. Dies kann mittels Skripten, die die Installation der Software umsetzen, geschehen. Auch Immutable Server können mit Technologien wie Docker genutzt werden. Auf Linux-Betriebssystemen ist ein Ausrollen mit Paketmanagern wie rpm (Red Hat), dpkg (Debian/Ubuntu) oder zypper (SuSE) auch möglich. Um mit Fehlern und daraus resultierenden Problemen umzugehen, existieren verschiedene Deployment-Strategien: Ein Rollback ist die Auslieferung einer vorherigen Version eines Microservices. Dabei müssen mögliche Datenbankänderungen berücksichtigt werden, da diese eine Rückkehr in den alten Zustand verhindern können. Alternativ können Fehler mit einem Roll Forward behoben werden, dem Ausrollen einer neuen, den Fehler nicht mehr enthaltenden Version. Continuous Deployment beschreibt die Strategie, jegliche Änderung sofort auf das Produktivsystem zu bringen, um die Anzahl an Änderungen pro Release klein zu halten. Mit Blue/Green-Deployments werden zwei parallele Microservice-Instanzen beschrieben, wodurch Änderungen auf einer Produktivumgebung wirksam werden, beobachtet werden können und in einem Fehlerfall das Routing zwischen der neuen und der alten Instanz gesteuert werden kann. [Wol16, S. 263 - 269]

2.3 Cloud-Infrastruktur

2.3.1 Definition

Unter Cloud-Computing versteht man das Nutzen von Computing-Ressourcen über das Netzwerk. Durch Virtualisierung können so Ressourcen eines von Cloud-Anbietern gehosteten Rechenzentrums gekapselt werden. In einer Cloud-Umgebung können Konzepte wie Continuous Delivery umgesetzt werden. [Ste20, S. 20]

Softwarecontainer sind eine Methode, um eine beliebige Anwendung in eine normierte, portable Einheit zu verpacken, die ein lauffähiges Gebilde ist. Die zentrale Eigenschaft einer solchen Umgebung ist es, dass der Container als Prozesskapsel von dem restlichen System isoliert ist und im Rahmen bestimmter Grenzen läuft. Dies erlaubt sowohl unterschiedliche Applikationen als auch verschiedene Komponenten derselben Applikation in separaten Containern zu betreiben. Ein Effekt davon ist, dass diese in der Cloud dann unabhängig voneinander skaliert werden können. Diese Modularisierung wird insbesondere bei Microservices als Prinzip für Entwicklung und Applikationsarchitektur auf die Spitze getrieben.

Für die Orchestrierung einer komplexen containerisierten Anwendung wird ein Cluster-Manager benötigt. Cluster bezeichnet dabei eine zusammengehörige Gruppe von Servern mit Containern, während die Instanzen, die in einem solchen Konstrukt eingebunden sind, als Knoten (Nodes) bezeichnet werden. Beispiele dafür sind Swarm Mode (Teil von Docker) oder Kubernetes.

[Ste20, S. 46-63]

2.3.2 Docker

- Docker (Kurzbeschreibung)
- Docker Compose
- Images
- Volumes

2.3.3 Kubernetes

Kubernetes ist ein moderner Standard für den verteilten Betrieb von containerisierten Applikationen auf Infrastrukturen aus der Public Cloud. Kubernetes nutzt Docker als Default-Container-Engine. Kubernetes besitzt eine eigene API, welche vor allem bei der Entwicklung von eigenen Werkzeugen genutzt wird. Das Tool kubectl spricht die Cluster über die auf dem Master laufende API an. Der Master steuert alle angeschlossenen Worker-Nodes und setzt die vom Anwender gewünschten Veränderungen um.

Objekte in Kubernetes sind eine Reihe von verschiedenen Dingen, die der Nutzer im Cluster

betreiben kann.

Deployement

Ein Deployement ist ein Objekt, welches etwas im Cluster installiert um es dort laufen zu lassen.

Pod

Pods sind kleine Organisationseinheiten, deren Aufgabe es ist, Container zu enthalten. Pods sind Objekte die vervielfältigt werden können, um die darin enthlaltene Appliaktion im Cluster zu skalieren.

Service

Rollout

Replica Set

Secret

Templates

[Ste20, S. 166-192]

2.4 Modellgetriebene Entwicklung von Microservices

Herausforderungen beim Domain-Driven Microservice Design:

- Ableitung von Microservices aus Domänenmodellen
- Fehlende Infrastrukturkomponenten in Domänenmodellen
- Autonome Domänenmodellierung

Mittel der modellgetriebenen Entwicklung zur Bewältigung von Herausforderungen

- Zwischenmodelle, die Microservice-Schnittstellen und -Bereitstellung spezifizieren.
- Reduzierung der Inoffizialität von Domänenmodellen durch Modellierungskonventionen
- Richtlinien für Autonome Domänenmodellierung

[RSS18]

Tools:

AjiL is an Eclipse-based tool for graphical MSA modeling. It features a diagram editor for modeling rudimentary domain models, functional and infrastructure microservices, and provided and required interfaces. Thus, AjiL facilitates creating intermediate models for service interfaces (see the section “Intermediate Models”). AjiL ships with a code generator that produces microservice implementations from AjiL models. The generated Java code is based on Spring Boot and Spring Cloud. It is directly runnable and comprises interface implementations, stubs for implementing services’ business logic, and configuration files for infrastructure components. However, AjiL currently does not support round-trip engineering.4 The tool is available at github.com/SeelabFhdo/AjiL. [RSS18]

Open Cloud Computing Interface (OCCI) is proposed as a community-based and open recommendation standard for managing any kind of cloud resources. Currently, only runtime implementations exist for OCCI, and each one targets a specific cloud service model such as Infrastructure as a Service (IaaS), Platform as a Service (PaaS), or Software as a Service (SaaS). Thus, OCCI lacks an approach to model and execute different OCCI artifacts. Our approach provides a generic modeling framework coupled with a generic runtime implementation. In this article, we propose the OCCIware approach, which represents the first approach to design, validate, generate, implement, deploy, execute, and supervise everything as a service with OCCI. This approach provides OCCIware Studio, the first model-driven tool chain for OCCI. It is built around OCCIware Metamodel, which defines the static semantics for the OCCI standard in Ecore and OCL. In addition, it proposes OCCIware Runtime, the first generic OCCI runtime implementation targeting all the cloud service models (IaaS, PaaS, and SaaS). OCCIware provides a unique and unified framework to manage OCCI artifacts and, at the same time, it represents a factory to build cloud domain-specific modeling frameworks where each framework targets a specific cloud domain. OCCIware has been applied in various cloud domains and use-cases, which validate its applicability. [ZCM19]

2.5 Noch zu sichten

2.5.1 ARGON - automated cloud infrastructure provisioning using a domain-specific language

Infrastructure as Code (IaC) is an approach to infrastructure automation based on practices from software development. IaC tools use scripts to specify the creation, update, and execution of cloud infrastructures. As each cloud provider offers a different type of infrastructure, the definition of an infrastructure element implies to write several lines of code that strongly depend on the target cloud provider. As a result, managing IaC scripts has become a time-consuming and error-prone activity. In previous work, we have presented ARGON, which is a model-driven infrastructure provisioning tool. ARGON implements a domain-specific language (DSL) for modeling the characteristics of the cloud infrastructure and provides transformation engines to automate the infrastructure provisioning for different cloud providers. In this demonstration, we show the use of ARGON for modeling and provisioning a load balancer in Amazon Web Services and Microsoft Azure. The load balancer distributes incoming application traffic across multiple virtual machines.

https://ieeexplore.ieee.org/abstract/document/8904494

2.5.2 model-driven approach for adaptive microservice-based systems using VxBPMN4MS

A microservice-based system is composed of numerous independently deployed and executed microservices, among which normally exist the complex dependencies. Traditional service composition approaches usually expect the business process predefined at design time. As a result, it is difficult for the microservice-based system to quickly adapt to the frequently changing operation environments and business requirements. To address the above limitations, we propose a variability-enabling and model-driven approach to developing adaptive microservice-based systems. Our approach first models the business process with variability using VxBPMN4MS, an extension of Business Process Model and Notation (BPMN) with support for variability, then transforms the business process model to variability supported microservice composition frameworks, and finally derives business process instances at run-time according to the predefined process configuration. We have developed a platform to automate the proposed approach as much as possible, and conducted a case study to evaluate the effectiveness of the proposed approach and platform.

https://ieeexplore.ieee.org/abstract/document/9529482

2.5.3 Context Mapper, a tool for DDD and microservice design.

Service-oriented architectures and microservices have gained much attention in recent years; companies adopt these concepts and supporting technologies in order to increase agility, scalability, and maintainability of their systems. Decomposing an application into multiple independently deployable, appropriately sized services and then integrating such services is challenging. With strategic patterns such as Bounded Context and Context Map, Domain-driven Design (DDD) can support business analysts, (enterprise) architects, and microservice adopters. However, existing architecture description languages do not support the strategic DDD patterns sufficiently; modeling tools for DDD primarily focus on its tactical patterns. As a consequence, different opinions on how to apply strategic DDD exist, and it is not clear how to combine its patterns. Aiming for a clear and concise interpretation of the patterns and their combinations, this paper distills a meta-model of selected strategic DDD patterns from the literature. It then introduces Context Mapper, an open source project that a) defines a Domain-specific Language (DSL) expressing the strategic DDD patterns and b) provides editing, validation, and transformation tools for this DSL. As a machine-readable description of DDD, the DSL provides a modeling foundation for (micro-)service design and integration. The models can be refactored and transformed within an envisioned tool chain supporting the continuous specification and evolution of Context Maps. Our validation activities (prototyping, action research, and case studies) suggest that the DDD pattern clarification in our meta-model and the Context Mapper tool indeed can benefit the target audience.

https://www.scitepress.org/Papers/2020/89105/89105.pdf

2.5.4 DOML simplifies infrastructure deployments by unifying multiple IaC languages

One of the main DevOps practices is the automation of resource provisioning and deployment of complex software. This automation is enabled by the explicit definition of Infrastructure-as-Code (IaC), i.e., a set of scripts, often written in different modelling languages, which defines the infrastructure and applications to be deployed.

We introduce the DevOps Modelling Language (DOML), a new Cloud modelling language for infrastructure deployments. DOML is a modelling approach that can be mapped into multiple IaC languages, addressing infrastructure provisioning, application deployment and configuration at once. The idea behind DOML is to use a single modelling paradigm which can help to reduce the need of deep technical expertise in using different specialised IaC languages.

We present the DOML's principles and discuss the related work on IaC languages. We demonstrate the DOML advantages for the end-user in comparison with state-of-the-art IaC languages such as Ansible, Terraform, and Cloudify, and show its effectiveness through an example.

Literaturverzeichnis

[Das17] Sebastian Daschner. *Architecting modern Java EE applications: designing lightweight, business-oriented enterprise applications in the age of cloud, containers, and Java EE 8*. Packt, Birmingham, 2017. Sebastian Daschner ; Foreword by: David Blevins, Founder and CEO, Tomitribe.

[Dow18] Herbert Dowalil. *Grundlagen des modularen Softwareentwurfs: der Bau langlebiger Mikro- und Makro-Architekturen wie Microservices und SOA 2.0*. Hanser, München, 2018. Herbert Dowalil.

[Eva07] Eric Evans. *Domain-driven design: tackling complexity in the heart of software*. Addison-Wesley, Upper Saddle River, NJ [u.a.], 11. printing edition, 2007. Eric Evans.

[New15] Sam Newman. *Microservices: Konzeption und Design*. mitp-Verl., [Frechen], 1. aufl. edition, 2015. Sam Newman. Übers. aus dem Amerikan. von Knut Lorenzen.

[RSS18] Florian Rademacher, Jonas Sorgalla, and Sabine Sachweh. Challenges of domain-driven microservice design: A model-driven perspective. *IEEE Software*, 35(3):36–43, 2018.

[Ste20] Daniel Stender. *Cloud-Infrastrukturen: das Handbuch für DevOps-Teams und Administratoren*. Rheinwerk computing. Rheinwerk Computing, Bonn, 1. auflage edition, 2020. Daniel Stender.

[Wol16] Eberhard Wolff. *Microservices: Grundlagen flexibler Softwarearchitekturen*. dpunkt.verlag, Heidelberg, 1., korrigierter nachdruck edition, 2016. Eberhard Wolff.

[ZCM19] Faiez Zalila, Stéphanie Challita, and Philippe Merle. Model-driven cloud resource management with occiware. *Future Generation Computer Systems*, 99:260–277, 2019.